SISTEMA DE GESTIÓN DE COWORKING Y RESERVAS

DOCUMENTACIÓN DEL DESARROLLO

Explicación Técnica del Sistema, Arquitectura y POO

Producto: Sistema de Gestión de Coworking y Reservas
Desarrolladores: Jonathan Lester Tullo, Maximiliano Scollo, David Samuel Ortega
Versión: 1.0 (Versión Funcional)
Año: 2026

ÍNDICE DE CONTENIDOS

1. Objetivo y Descripción del Sistema **3**
2. Tecnologías, Herramientas y Lenguajes Utilizados **3**
3. Organización del Trabajo en Equipo **4**
4. Arquitectura del Sistema por Capas **4**
4.1. Capa de Presentación (Windows Forms) 4
4.2. Capa de Negocio (Gestoras y Reglas) 4
4.3. Capa de Modelos (Entidades de Dominio) 4
4.4. Capa de Conexión (ConexionDB) 4
4.5. Capa de Persistencia (MySQL) 4
5. Programación Orientada a Objetos y Clases C# **5**
5.1. Conexión centralizada (Conexion.cs) 5
5.2. Interfaz de disponibilidad (IReservable.cs) 5
5.3. Enumerador de estado (EstadoReserva.cs) 5
5.4. Clase base abstracta (Recurso.cs) 5
5.5. Herencia y Polimorfismo (Sala, Oficina, Puesto) 5
5.6. Clases de negocio (Cliente.cs y Reserva.cs) 6
5.7. Gestión en memoria y persistencia (Gestora / DAO) 6
6. Manejo de Errores y Excepciones (MySQL + C#) **6**
7. Estado Actual de Avance y Requerimientos Evolutivos **7**
8. Instrucciones para la Ejecución del Proyecto **7**

1. Objetivo y Descripción del Sistema

Este proyecto consiste en el desarrollo de una aplicación de escritorio programada en C# con Windows Forms, conectada de manera centralizada a una base de datos MySQL alojada en el entorno de XAMPP. El objetivo principal es administrar de forma integral un espacio de coworking, permitiendo la gestión de clientes, el catálogo de recursos alquilables (salas de reunión, oficinas privadas y puestos de trabajo) y el control de reservas por fecha y horario.

El sistema resuelve los problemas operativos fundamentales del negocio: asegura que no existan reservas duplicadas para un mismo recurso en el mismo horario mediante validaciones estrictas en la base de datos, calcula automáticamente los costos de alquiler aplicando tarifas base y recargos por comodidades, y mantiene un registro automático de auditoría para cada cancelación.

2. Tecnologías, Herramientas y Lenguajes Utilizados

La solución se apoya en un stack tecnológico moderno, robusto y estandarizado para entornos de escritorio:

- **C# (.NET Framework 4.7.2):** Lenguaje principal orientado a objetos con tipado estático, herencia, interfaces, colecciones genéricas y LINQ.
- **Windows Forms (WinForms):** Framework para la interfaz gráfica de usuario en Windows (formularios FrmLogin y FrmPrincipal con DataGridView, ComboBox y TabControl).
- **MySQL / MariaDB (v10.4+ en XAMPP):** Motor relacional para la persistencia centralizada en localhost:3306 (integridad referencial, vistas, triggers de auditoría, funciones y procedimientos almacenados con SIGNAL 45000).
- **MySql.Data:** Controlador oficial de Oracle para ADO.NET (v9.5.0), que administra las conexiones MySqlConnection y comandos MySqlCommand.
- **Microsoft Visual Studio 2022:** Entorno de desarrollo integrado utilizado para la compilación, diseño visual, gestión de dependencias NuGet y depuración del código.
- **Git y GitHub:** Sistema de control de versiones distribuido y plataforma en la nube para el trabajo coordinado en equipo y control de cambios.
- **MySQL Workbench y phpMyAdmin:** Herramientas visuales de administración para la ejecución del script SQL (DB_TPFinal.sql) y verificación de datos.
- **WhatsApp API / Web:** Protocolo URI de enlace directo (wa.me) para derivar consultas técnicas a la línea de soporte del coworking.

3. Organización del Trabajo en Equipo

El desarrollo del proyecto se organizó de manera modular entre los tres integrantes del equipo, coordinando cambios mediante control de versiones en Git/GitHub y documentando cada etapa técnica:

- **Jonathan Lester Tullo (shonimdp@gmail.com):** Diseño y creación de la base de datos MySQL en XAMPP (tablas, relaciones, procedimientos almacenados, vistas, funciones y trigger). Creación de la solución base en Visual Studio, instalación de dependencias NuGet (MySql.Data), programación de la clase centralizada Conexion.cs y diseño de la clase abstracta base Recurso.cs junto con la interfaz IReservable.cs.
- **Maximiliano Scollo (maximilianoscollo17@gmail.com):** Programación de las clases derivadas (Sala.cs, OficinaPrivada.cs y PuestoTrabajo.cs) implementando el cálculo polimórfico de costos y disponibilidad. Implementación de los modelos Cliente.cs y Reserva.cs, programación de la clase GestoraCoworking.cs con colecciones en memoria y actualización del diagrama Entidad-Relación (DER).
- **David Samuel Ortega (davidortegaa86@gmail.com):** Diseño completo de la interfaz de usuario en Windows Forms, desarrollo del formulario de autenticación (FrmLogin) con control de credenciales y asistencia técnica vía WhatsApp, preparación de recursos gráficos y maquetado del formulario principal (FrmPrincipal) con solapas para Control, Reservas y Gestión administrativa.

4. Arquitectura del Sistema por Capas

Para garantizar la separación de responsabilidades y evitar que los formularios concentren toda la lógica del sistema, la aplicación se organizó en las siguientes capas:

- **4.1. Capa de Presentación (Windows Forms):** Formularios FrmLogin y FrmPrincipal. Su función es interactuar con el usuario, capturar eventos, validar entradas y presentar la información en controles adecuados (DataGridView, ComboBox, DateTimePicker).
- **4.2. Capa de Negocio (Gestoras y Reglas):** La clase GestoraCoworking.cs administra las listas en memoria y coordina las reglas del negocio, aplicando búsquedas y filtrados de reservas por fecha, cliente o estado.
- **4.3. Capa de Modelos (Entidades de Dominio):** Representan las entidades del dominio de la aplicación: Cliente, Reserva, Recurso y sus clases derivadas especializadas.
- **4.4. Capa de Conexión (ConexionDB):** La clase estática Conexion.cs centraliza los parámetros de conexión hacia el servidor MySQL en localhost:3306, entregando conexiones que se abren puntualmente bajo bloques using.
- **4.5. Capa de Persistencia (MySQL):** La base de datos MySQL asegura la consistencia de los datos en disco, encapsulando operaciones críticas mediante procedimientos almacenados, funciones de verificación y triggers de auditoría.

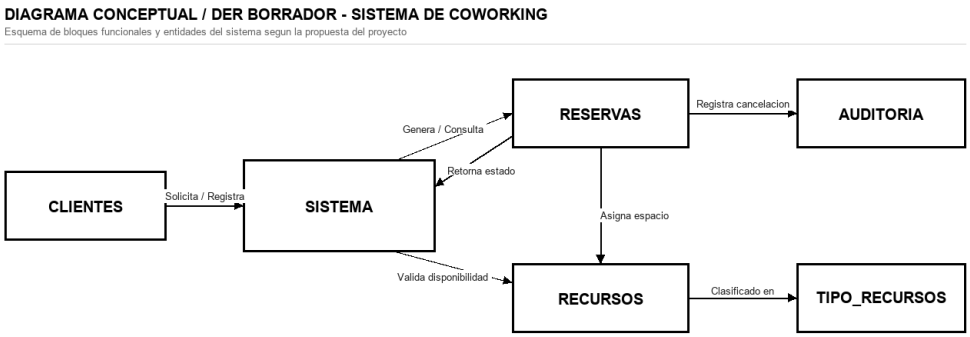


Figura 1: Diagrama Conceptual y Bloques del Sistema (Borrador)

5. Programación Orientada a Objetos y Clases C#

A continuación se detallan las decisiones de diseño adoptadas en la implementación orientada a objetos:

5.1. Conexión centralizada (Conexion.cs)

La clase Conexion es de tipo static porque no representa una entidad del negocio sino un servicio utilitario de infraestructura. Su método CrearConexion() entrega una instancia configurada de MySqlConnection pero no ejecuta .Open() de forma anticipada. Cada formulario o método abre la conexión cuando la requiere mediante la instrucción using, garantizando que los recursos se liberen inmediatamente al concluir la operación.

5.2. Interfaz de disponibilidad (IReservable.cs)

Define el contrato bool PuedeReservarse(DateTime fecha). El uso de una interfaz permite desacoplar la capacidad de reserva de la jerarquía de herencia: cualquier elemento presente o futuro del coworking que deba verificar disponibilidad implementa este contrato sin requerir modificaciones en el resto del sistema.

5.3. Enumerador de estado (EstadoReserva.cs)

Contiene los valores tipados: Pendiente, Confirmada y Cancelada. Evita el uso de cadenas de texto literales en el código fuente, previniendo errores de tipeo y asegurando compatibilidad con la columna VARCHAR de la base de datos mediante .ToString().

5.4. Clase base abstracta (Recurso.cs)

Recurso es una clase abstracta porque en el dominio del problema no se reservan recursos genéricos, sino tipos específicos de espacios. Reúne los atributos y propiedades comunes (IdRecurso, Nombre, Descripcion, Ubicacion, PrecioPorHora, IdTipoRecurso) y define la

firma abstracta CalcularCosto(int horas), forzando a que cada clase derivada implemente su propia regla de facturación.

5.5. Herencia y Polimorfismo (Sala, Oficina, Puesto)

Las clases derivadas especializan el comportamiento base incorporando características propias:

- **Sala:** Incorpora la propiedad CantSillas. En CalcularCosto, multiplica la tarifa base por las horas y añade un adicional por mantenimiento ($8.000 para salas de más de 30 sillas y $16.000 para más de 60).
- **OficinaPrivada:** Incorpora la propiedad booleana MobiliarioIncluido. Si la oficina cuenta con equipamiento completo, añade un adicional fijo de $1.500 al valor del alquiler.
- **PuestoTrabajo:** Incorpora la propiedad NroCategoria (1, 2 o 3). Aplica un plus tarifario según el nivel de comodidades del box ($3.500 para categoría 1, $5.500 para categoría 2 y $8.000 para categoría 3).

5.6. Clases de negocio (Cliente.cs y Reserva.cs)

Cliente modela los datos del usuario (IdCliente, Nombre, Apellido, Telefono, Mail, FechaRegistro).

Reserva actúa como entidad relacional que vincula al cliente con el recurso, la fecha y hora asignadas, la duración, el total calculado y el estado correspondiente.

5.7. Gestión en memoria y persistencia (Gestora / DAO)

GestoraCoworking administra colecciones genéricas List<Reserva> en memoria para realizar consultas, modificaciones y cancelaciones. En una etapa evolutiva posterior, las operaciones de persistencia serán trasladadas progresivamente a clases específicas de acceso a datos (GestorDAO) para completar el desacoplamiento total de los formularios.

6. Manejo de Errores y Excepciones (MySQL + C#)

Para salvaguardar la consistencia en operaciones críticas de base de datos, el sistema implementa una doble estrategia de control de excepciones:

- **1. Validación en Base de Datos:** El procedimiento almacenado sp_alta_reserva evalúa mediante fn_verificar_disponibilidad si el recurso se encuentra ocupado calculando el intervalo exacto (fecha y duración en horas). De existir superposición horaria, la base interrumpe la transacción ejecutando SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El recurso ya se encuentra reservado u ocupado en ese intervalo de horario.', impidiendo la inserción de registros duplicados o solapados.
- **2. Captura Específica en C#:** En el código C#, las llamadas a MySQL se protegen mediante bloques catch (MySqlException ex). La aplicación examina si ex.SqlState ==

"45000" para capturar el error de negocio y presentar al usuario un mensaje informativo claro en pantalla, diferenciándolo de fallas de infraestructura o red.

- **3. Errores Generales:** Se incorpora un bloque secundario catch (Exception ex) para capturar cualquier anomalía no prevista, evitando en todos los casos el uso de bloques catch vacíos.

7. Estado Actual de Avance y Requerimientos Evolutivos

Conforme al desarrollo progresivo por etapas planificado para el sistema:

- **Etapas Implementadas (1 a 4):** Base de datos MySQL completa con integridad referencial, procedimientos almacenados, vistas, función de verificación y trigger de auditoría. Modelos C# completos con herencia, clases abstractas, interfaces y polimorfismo. Formularios Windows Forms funcionales con autenticación de usuarios, roles de acceso y disposición visual terminada.
- **Etapas Evolutivas Planificadas (5 y 6):** Separación definitiva de clases DAO de acceso a datos (Etapa 5), incorporación de importación/exportación de catálogos y reservas en formato JSON y configuración externa de la cadena de conexión (Etapa 6).

8. Instrucciones para la Ejecución del Proyecto

Para ejecutar la aplicación en un entorno de desarrollo o prueba local:

- **1. Servidor de Base de Datos:** Iniciar el panel de XAMPP y activar el servicio MySQL en el puerto 3306.
- **2. Carga de la Base:** Abrir MySQL Workbench o phpMyAdmin y ejecutar el script ubicado en DB/DB_TPFinal.sql. Esto creará la base de datos tp_final_db junto con sus tablas, vistas, funciones, procedimientos y datos iniciales de prueba.
- **3. Compilación y Ejecución:** Abrir la solución SistemaDeGestion.slnx en Visual Studio 2022, verificar que las referencias NuGet a MySql.Data se encuentren resueltas y presionar F5 (Iniciar depuración).